FlexER: Flexible Entity Resolution for Multiple Intents

BAR GENOSSAR, Technion – Israel Institute of Technology, Israel
ROEE SHRAGA, Northeastern University, MA, USA
AVIGDOR GAL, Technion – Israel Institute of Technology, Israel

Entity resolution, a longstanding problem of data cleaning and integration, aims at identifying data records that represent the same real-world entity. Existing approaches treat entity resolution as a universal task, assuming the existence of a **single** interpretation of a real-world entity and focusing only on finding matched records, separating corresponding from non-corresponding ones, with respect to this single interpretation. However, in real-world scenarios, where entity resolution is part of a more general data project, downstream applications may have varying interpretations of real-world entities relating, for example, to various user needs. In what follows, we introduce the problem of *multiple intents entity resolution* (MIER), an extension to the universal (single intent) entity resolution task. As a solution, we propose FlexER, utilizing contemporary solutions to universal entity resolution tasks to solve multiple intents entity resolution. FlexER addresses the problem as a multi-label classification problem. It combines intent-based representations of tuple pairs using a multiplex graph representation that serves as an input to a graph neural network (GNN). FlexER learns intent representations and improves the outcome to multiple resolution problems. A large-scale empirical evaluation introduces a new benchmark and, using also two well-known benchmarks, shows that FlexER effectively solves the MIER problem and outperforms the state-of-the-art for a universal entity resolution.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: entity resolution, entity matching, graph neural networks, supervised learning

ACM Reference Format:
Bar Genossar, Roee Shraga, and Avigdor Gal. 2023. FlexER: Flexible Entity Resolution for Multiple Intents. *Proc. ACM Manag. Data* 1, 1, Article 42 (May 2023), 27 pages. https://doi.org/10.1145/3588722

1 INTRODUCTION

An essential component of any data science lifecycle involves data preparation. Accordingly, contemporary data projects intensively invest in large-scale data cleaning and integration techniques to combine data from multiple heterogeneous sources into meaningful and valuable information. Consider, for example, a data-intensive organization (*e.g.*, an online shopping company) that continuously collects data on user queries to better understand their needs. The (mostly implicit) intents of users may vary. While some users are interested in basketball shoes, regardless of their brand, others may be more interested in buying branded products, caring less whether shoes are designated as running shoes.

At the heart of the data preparation realm lies the *entity resolution* task (with variations of entity matching, record linkage, deduplication, and more), which has been extensively studied over the

Authors' addresses: Bar Genossar, Technion – Israel Institute of Technology, Haifa, Israel, sbargen@campus.technion.ac.il; Roee Shraga, Northeastern University, Boston, MA, USA, r.shraga@northeastern.edu; Avigdor Gal, Technion – Israel Institute of Technology, Haifa, Israel, avigal@technion.ac.il.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART42 $15.00
https://doi.org/10.1145/3588722

Table 1. Amazon Product Dataset Sample (D_{ex})

r_{id}	**Product title**
r_1	Nike Men's Lunar Force 1 Duckboot
r_2	NIKE Men Lunar Force 1 Duckboot, Black/Dark Loden-BROGHT Crimson
r_3	NIKE Men's Air Max Stutter Step Ankle-High Basketball Shoe
r_4	Nike Men's Air Max 2016 Running Shoe
r_5	adidas Performance Men's D Rose 6 Boost Primeknit Basketball
r_6	The Man Who Tried to Get Away

past decades (see books and surveys [8, 13, 17]). Entity resolution is a data integration task that aims at identifying same real-world entities that are represented by different data records (instances). Specifically, entity resolution can be used to "clean" a dataset from duplicate tuples referring to the same entity, offering a useful post processing tool for integrating multiple data sources.

Entity resolution techniques for resolving duplicates in a dataset were developed over years of research. Traditional methods focused on string similarity [22, 23, 32, 37] and rule-based methods [54, 55]. Learning-based approaches were also suggested [3, 31], followed by deep learning methods in recent years [15, 16, 25, 28, 34, 41, 64]. Following a common practice for text processing, the use of *pre-trained language models*, and specifically, BERT-based models [10], was also introduced for entity resolution [5, 33, 35, 46].

The aforementioned methods share a common assumption regarding the universality of the entity resolution problem, assuming the existence of a **single** interpretation to the notion of a real-world entity. In particular, most solutions aim to create a single clean view of a dataset, by separating corresponding from non-corresponding record pairs, with respect to this single interpretation.

The universal property of entity resolution solutions is being challenged by the development of applications that require the ability to provide a fine-grained (or personalized) analysis, offering services that are tailored to specific user needs and requirements [4, 62]. For example, online shops aim at personalizing shopping experience to the needs of individual users, having to infer such needs from mainly implicit feedback [29], such as clicks and mouse movements. In such a setting, having a universal resolution facility is likely to cripple the resolution process, failing to cater to the needs of all applications at all times. To illustrate the need for a non-universal (flexible) resolution, we use the *AmazonMI* dataset, a newly suggested benchmark based on amazon products data (see Section 5.1), in the following motivating example.

1.1 Motivating Example: From Record Duplication to Entity Resolution Interpretation

Table 1 depicts an excerpt of the AmazonMI dataset to illustrate the differences between the universal approach to entity resolution and the flexible entity resolution we present and address in this work. Figure 1 offers visual illustration to the multiplicity of entity interpretations in the example.

Record duplication is usually the result of discordant representations (*e.g.,* multi-lingual, synonyms, capitalizations), changes in the data over time, typos, *etc.* For example, records r_1 and r_2 in Table 1 refer to the same pair of Nike basketball shoes called "Men's Lunar Force 1 Duckboot." The difference between r_1 and r_2 originates from capitalization issues (Nike vs. NIKE) and additional specification (*e.g.,* color). Contrarily, r_1 and r_6 will most likely be conceived as different, as the latter refers to a book.

The pair (r_1, r_3) is an example of a pair that, under a certain interpretation, does not refer to the same entity, representing two different Nike basketball shoe variants. Yet, under a different interpretation, both r_1 and r_3 are Nike basketbsall shoes. Similarly, r_1 differs from r_4 and r_5 since they represent a different type of Nike shoes (basketball vs. running) and basketball shoes of different brands (Nike vs. Adidas), respectively, but may be determined to be representatives of the same entity (Nike shoes and basketball shoes, respectively) under a broader interpretation. Such different interpretation may be attributed to different user tastes or different contexts. For example, a pro basketball player would be more sensitive to differences between basketball shoes than a user that seeks shoes for a neighborhood afternoon fun game.

Over the years, entity resolution solutions resolved a single entity interpretation, detecting r_1 and r_2 as duplicates, to create a (single) clean dataset, *e.g.,* by only preserving r_1. The multiple intent phenomenon challenges any downstream application that requires entity matching and involves personalization to user's needs. Beyond online shopping, multiple intents can be frequently found in the domain of recommendation systems, where user's implicit feedback (*e.g.*, item selection and search queries) can serve in understanding a user's intent for better responding to her needs [29].

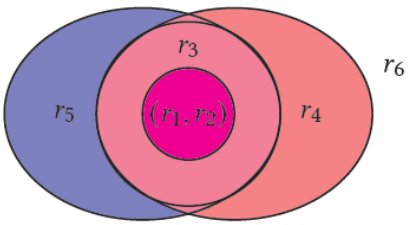


Fig. 1. Possible entity resolution solutions for different entity interpretations over the tuples from Table 1.

1.2 Main Contributions

The motivating example offers an intuitive description of a scenario where a single dataset may serve as a basis for multiple clean views to be generated by an entity resolution solution, suggesting different interpretations of an entity in each such view. We use the term *intent* (formally defined in Section 2.2) to reflect user preferences that, in the scope of this paper, relate to the interpretation of the output view. We are, in particular, interested in intents that are unknown apriori and therefore cannot be constructed using data that exists in the database. Rather, such intents are known only through the training set that labels tuple pairs to be matched under a given interpretation. A typical such scenario can be found in recommendation systems where feedback is collected from users, either implicitly or explicitly, to be used to indicate intent. Another possible scenario may be motivated using machine learning algorithm for entity matching, where different tuning parameters yield different outcomes even for the same algorithm. For example, see the use-case of Yad VaShem, the Jewish Holocaust museum in Jerusalem [51].

In this work, we define the problem of *multiple intents entity resolution* (MIER), an extension to the universal (single intent) entity resolution task. The MIER task considers multiple intents when creating a solution for potential downstream applications that involve entity resolution problems. In the absence of human interpretation to intents (recall the use of implicit feedback in recommendation systems to indicate intent), we aim at training a model to offer such interpretation based on training data and intent cross learning. We propose FlexER, a solution to the MIER problem that utilizes contemporary solutions to universal entity resolution tasks (*e.g.,* DITTO [35]). FlexER positions the problem as a multi-label classification problem and combines intent-based

Ground Truth Inference for Weakly Supervised Entity Matching

RENZHI WU, Georgia Institute of Technology, USA
ALEXANDER BENDECK, Georgia Institute of Technology, USA
XU CHU, Georgia Institute of Technology, USA
YEYE HE, Microsoft Research, USA

Entity matching (EM) refers to the problem of identifying pairs of data records in one or more relational tables that refer to the same entity in the real world. Supervised machine learning (ML) models currently achieve state-of-the-art matching performance; however, they require a large number of labeled examples, which are often expensive or infeasible to obtain. This has inspired us to approach data labeling for EM using *weak supervision*. In particular, we use the labeling function abstraction popularized by Snorkel, where each labeling function (LF) is a user-provided program that can generate many noisy match/non-match labels quickly and cheaply. Given a set of user-written LFs, the quality of data labeling depends on a *labeling model* to accurately infer the ground-truth labels. In this work, we first propose a simple but powerful labeling model for general weak supervision tasks. Then, we tailor the labeling model specifically to the task of entity matching by considering the EM-specific transitivity property.

The general form of our labeling model is simple while substantially outperforming the best existing method across ten general weak supervision datasets. To tailor the labeling model for EM, we formulate an approach to ensure that the final predictions of the labeling model satisfy the transitivity property required in EM, utilizing an exact solution where possible and an ML-based approximation in remaining cases. On two single-table and nine two-table real-world EM datasets, we show that our labeling model results in a 9% higher F1 score on average than the best existing method. We also show that a deep learning EM end model (DeepMatcher) trained on labels generated from our weak supervision approach is comparable to an end model trained using tens of thousands of ground-truth labels, demonstrating that our approach can significantly reduce the labeling efforts required in EM.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: entity matching, weak supervision, labeling model, transitivity

ACM Reference Format:
Renzhi Wu, Alexander Bendeck, Xu Chu, and Yeye He. 2023. Ground Truth Inference for Weakly Supervised Entity Matching. *Proc. ACM Manag. Data* 1, 1, Article 32 (May 2023), 28 pages. https://doi.org/10.1145/3588712

1 INTRODUCTION

Entity matching (EM) refers to the process of determining if a pair of records from two data sources refer to the same real-world entity. EM has many applications, for example, in matching product listings for competitive pricing [2] and in building knowledge graphs [28]. As a long-standing problem, EM has been extensively studied (e.g., see surveys [18, 29, 30, 34, 36, 76]).

Authors' addresses: Renzhi Wu, Georgia Institute of Technology, Atlanta, USA, renzhiwu@gatech.edu; Alexander Bendeck, Georgia Institute of Technology, Atlanta, USA, abendeck3@gatech.edu; Xu Chu, Georgia Institute of Technology, Atlanta, USA, xu.chu@cc.gatech.edu; Yeye He, Microsoft Research, Redmond, USA, yeyehe@microsoft.com.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART32 $15.00
https://doi.org/10.1145/3588712

The Need for Weak Supervision in EM. The most important requirement for EM solutions is high matching quality as it directly affects downstream application performance. State-of-the-art EM solutions that report the highest matching quality [29, 50, 59, 60] are supervised machine learning (ML) approaches that train a binary classifier to predict the label (match or non-match) for any tuple pair. However, these models require large numbers of labeled tuple pairs, which are often not available or costly to obtain [29]. The high human cost in data labeling has become a main bottleneck in adopting high-quality EM solutions in practice.

To meet the needs of data-hungry supervised ML models, ML practitioners have increasingly turned to *weak supervision* methods, in which a larger volume of cheaply generated, but often noisier, labeled examples is used in lieu of hand-labeled examples. Different forms of weak supervision have been investigated, including the use of non-expert crowd workers [33], pretrained models [24], and rules/patterns/heuristics [75].

To unify different forms of supervision, the *data programming* paradigm [69] has been proposed, in which users write *labeling functions* (LFs) to programmatically label training data, rather than manually labeling each example by hand. Each LF is a small user-provided program (e.g., in Python) that leverages noisy signals in the data or domain knowledge to provide a label (or abstain) for an input example. In binary classification tasks such as EM, the output of each LF is +1 (positive class), -1 (negative class), or 0 (abstain). The data programming approach was first implemented in Snorkel [66], a weakly supervised data labeling system. Snorkel allows users to write LFs and then uses a generative model to combine all LFs to produce probabilistic labels. The data programming approach has been widely adopted in various ML tasks [31, 32, 53, 54, 73, 81, 85].

Example Labeling Functions. The LFs shown in Figure 1 are two user-written LFs developed using an existing tool [84] for matching electronic products [1]. The first LF, "name_overlap", encodes the intuition that matching products should have similar "name" attributes. Specifically, pairs of products with sufficiently high/low word overlap in their names are predicted as matches (+1)/non-matches (-1). When the amount of word overlap does not provide conclusive evidence either way, the LF abstains (0). The second LF, "size_unmatch", uses a regular expression to search the screen size in patterns like "... Samsung **40'** LCD ...". When the screen sizes of a pair of products are different, the LF predicts the pair to be non-match (-1); otherwise, the LF abstains (0).

```
@labeling_function
def name_overlap(tpl_l, tpl_r):
    x = lower(tpl_l.name)
    y = lower(tpl_r.name)
    x = set(splitBySpace(x))
    y = set(splitBySpace(y))
    itsect = x.intersection(y)
    max_len = max(len(x), len(y))
    score = len(itsect) / max_len
    if score < 0.1:
        return -1
    elif score > 0.8:
        return 1
    else:
        return 0
```

```
@labeling_function
def size_unmatch(tpl_l, tpl_r):
    l_name_des = str(tpl_l.name) + " " \
                 + str(tpl_l.description)
    r_name_des = str(tpl_r.name) + " " \
                 + str(tpl_r.description)
    x = re.findall(r"[\d\.]+\'", l_name_des)
    y = re.findall(r"[\d\.]+\'", r_name_des)
    if len(x) == 0 or len(y) == 0:
        return 0
    x, y = x[0], y[0]
    if x != y:
        return -1
    else:
        return 0
```

Fig. 1. Two user-written LFs for the abt-buy dataset [1].

The predictions of multiple LFs can then be used to construct a *labeling matrix* X, where each row corresponds to a tuple pair, and each column corresponds to predictions of one LF for all tuple pairs. After constructing the labeling matrix, a *labeling model* is used to consolidate the labeling function predictions into a final label for each tuple pair. Figure 2 shows one instance of a labeling matrix with votes from LFs (LF1, LF2, ...) and inferred labels from a naive labeling model (+1 for match, -1 for non-match) as well as the unknown ground truth (GT) labels for each tuple pair.

Given a set of user-written LFs, the quality of data labeling depends on the *labeling model* that infers the ground-truth labels from the LFs. In this paper, we first present an embarrassingly simple

	LF1	LF2	LF3	...	Naïve Inferred Label	GT
(t_1, t_2)	1	1	1	...	1	1
(t_1, t_3)	1	1	-1	...	1	-1
(t_1, t_4)	1	1	0	...	1	1
...	...	...	...	...	...	...
(t_2, t_3)	0	-1	-1	...	-1	-1
(t_2, t_4)	0	0	-1	...	-1	1

Fig. 2. Example of a labeling matrix

yet highly effective labeling model (SIMPLE) for general weak supervision tasks. We then specialize the labeling model (SIMPLE-EM) for the task of EM by considering the transitivity property.

(1) Labeling Model for General Weak Supervision Tasks: Existing methods are mostly generative models, i.e., they model the process of generating the predictions of each LF from the hidden ground-truth labels [26, 32, 52, 68]. Under an assumed generative process, one could measure the likelihood of observing all LFs (i.e., the labeling matrix) and an estimation of the ground-truth labels can be obtained by maximizing the likelihood [26, 32, 52, 68]. A fundamental limitation of these approaches is the need to assume that the generative process takes certain simplified forms for ease of modeling or to make the model mathematically solvable. Such assumptions include the Markov assumption [26, 32, 68] and the mixture of independent sub-types assumption [52]. Evidently, these assumptions can often break in practice. Recent benchmark studies [86] suggest that the best existing labeling model for general weak supervision tasks is only 1.5% better than majority vote, and the second best method is already worse than majority vote. In addition, even with these various assumptions to simplify the generative process, the existing approaches are typically quite complicated and difficult to implement.

We take a fundamentally different approach that is a significant departure from the conventional wisdom that generative models should be used in labeling models. The core intuition of our method is that *labeling models are functions*. Conceptually, every labeling model takes in the labeling matrix X and predicts the label vector $\hat{\boldsymbol{y}}$. For example, X could be the sub-matrix in Figure 2 with columns of all LFs and $\hat{\boldsymbol{y}}$ could be the naive inferred label column in Figure 2. Designing a labeling model is to design a function G parameterized by Θ, and $\hat{\boldsymbol{y}} = G(X, \Theta)$ would be the predicted labels of the model. The prediction of the whole matrix X can be made by independently predicting each individual row/data point; i.e., $\hat{y_i} = g(X[i, :], \theta)$ where θ is the parameter of g. Different designs of existing labeling models can be seen as the function g taking different forms.

As discussed, existing labeling models typically build complicated models (e.g., probabilistic graphical models [27, 41, 55] and matrix completion models [38, 67]) by making various assumptions to handcraft g. In this paper, we ask a fundamental question that is contrary to the conventional wisdom: *Can we use a generic classifier (e.g., random forest) as g to be the labeling model and avoid all the complicated designs in the existing approaches?* We provide an affirmative answer to this question by presenting an embarrassingly simple method (in Section 3) that achieves better performance than the existing approaches across ten datasets.

(2) EM-specific Labeling Model with Transitivity: The *transitivity property of EM* states that for any three tuples t_i, t_j, and t_k, if t_i matches t_j and also matches t_k, then t_j has to match t_k. Concretely, consider the scenario in Figure 2 – a naive model would likely predict the tuple pair (t_2, t_4) as non-match, despite strong evidence that both (t_1, t_2) and (t_1, t_4) should be matches, which results in inconsistent label assignments that violate transitivity. Our goal is to explicitly model transitivity using constraints, so that matching decisions can be made in a holistic manner.

A naive approach to enforce transitivity is to do a post-processing step, which is sub-optimal. Intuitively, the additional signals from the transitivity constraints should be made an integral part of the labeling model to improve its accuracy. We thus attempt to incorporate transitivity directly in the labeling model. However, this makes the labeling model complex and difficult to solve.

Progressive Entity Matching: A Design Space Exploration

JAKUB MACIEJEWSKI, National and Kapodistrian University of Athens, Greece
KONSTANTINOS NIKOLETOS, National and Kapodistrian University of Athens, Greece
GEORGE PAPADAKIS, National and Kapodistrian University of Athens, Greece
YANNIS VELEGRAKIS, University of Trento & Utrecht University, The Netherlands

Entity Resolution (ER) is typically implemented as a batch task that processes all available data before identifying duplicate records. However, applications with time or computational constraints, e.g., those running in the cloud, require a progressive approach that produces results in a pay-as-you-go fashion. Numerous algorithms have been proposed for Progressive ER in the literature. In this work, we propose a novel framework for Progressive Entity Matching that organizes relevant techniques into four consecutive steps: (i) filtering, which reduces the search space to the most likely candidate matches, (ii) weighting, which associates every pair of candidate matches with a similarity score, (iii) scheduling, which prioritizes the execution of the candidate matches so that the real duplicates precede the non-matching pairs, and (iv) matching, which applies a complex, matching function to the pairs in the order defined by the previous step. We associate each step with existing and novel techniques, illustrating that our framework overall generates a superset of the main existing works in the field. We select the most representative combinations resulting from our framework and fine-tune them over 10 established datasets for Record Linkage and 8 for Deduplication, with our results indicating that our taxonomy yields a wide range of high performing progressive techniques both in terms of effectiveness and time efficiency.

CCS Concepts: • **Information systems → Entity resolution**.

Additional Key Words and Phrases: Progressive Entity Resolution, Nearest Neighbor Search, Blocking, Sorted Neighborhood

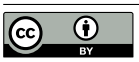
ACM Reference Format:
Jakub Maciejewski, Konstantinos Nikoletos, George Papadakis, and Yannis Velegrakis. 2025. Progressive Entity Matching: A Design Space Exploration. *Proc. ACM Manag. Data* 3, 1 (SIGMOD), Article 65 (February 2025), 25 pages. https://doi.org/10.1145/3709715

1 Introduction

Entity Resolution (ER), often also referred to as Record Linkage (a.k.a. Clean-clean ER) or Deduplication (a.k.a. Dirty ER), is a fundamental task in data management [24]. It deals with the challenge of identifying and linking data structures, typically referred to as *entity profiles*, that represent the same real-world object [7] The linked structures are referred to as duplicates. Detecting the duplicates is crucial for boosting the performance of a wide range of data management tasks, from recommendation to question answering.

An ER application in order to be successful, needs to overcome a series of challenges [3, 7, 13]. The first is *Volume*. ER algorithms should scale to thousands or even millions of entity profiles. This isn't straightforward, due to the the inherently quadratic nature of ER, which is why it is

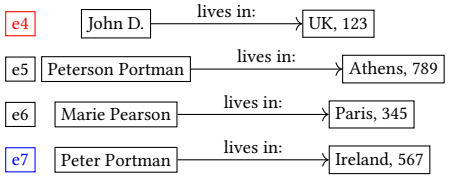
Authors' Contact Information: Jakub Maciejewski, National and Kapodistrian University of Athens, Greece, sdi1700080@di.uoa.gr; Konstantinos Nikoletos, National and Kapodistrian University of Athens, Greece, k.nikoletos@di.uoa.gr; George Papadakis, National and Kapodistrian University of Athens, Greece, gpapadis@di.uoa.gr; Yannis Velegrakis, University of Trento & Utrecht University, The Netherlands, i.velegrakis@uu.nl.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2836-6573/2025/2-ART65
https://doi.org/10.1145/3709715

id	First name	Last name	City	Zip Code
e1	John	Doe	London	123
e2	Peter	Portman	Dublin	567
e3	Donald	Hirsh	New Jersey	486

(a) Data source D_1



(b) Data source D_2

(e1, e4), (e2, e7), (e2, e5), ..., (e3, e6)

(c) Ordered set of candidate pairs

Fig. 1. Two clean data sources and the candidate pairs generated by a Progressive Entity Matching approach.

addressed through filtering [32]. The second is *Variety*. ER should seamlessly apply to data sources that vary in format, schema and structure. Yet, most ER approaches are crafted either for structured, e.g., relational data or semi-structured, e.g., RDF [29]. The third is *Velocity*. Most ER solutions operate in an offline mode, i.e., they produce results only after having processed all the input data. Unfortunately, there are cases where there are restrictions and cost in computational or temporal resources. An example is the cloud environment. In these cases, partial results are required within a specific time frame [9–11, 45].

To address these challenges, we focus on *Schema-agnostic Progressive Entity Matching*. Velocity is addressed by the progressive functionality, which yields results before processing all input data through a pay-as-you-go functionality. Volume is addressed by Filtering, which restricts the computational cost to the most similar entity profiles, disregarding those dissimilar. Variety is addressed by the schema-agnostic functionality, which represents every entity profile through a concatenation of all attribute values, regardless the respective attribute names.

Consider Figure 1 that illustrates a Record Linkage case, i.e., a case of two data sources D_1 and D_2, each one being duplicate-free, meaning that it contains no matching entity profiles, but there are duplicates across the sources. For instance, e_1 matches to e_4 and e_2 to e_7. The two sources need to be merged, and to do so, the duplicates need to be detected. Furthermore, the two data sources exhibit high variety, varying both in format (structured vs semi-structured) and in schema. Typically, an ER process consists of three steps [3, 13]: Filtering, Entity Matching and Clustering. Yet, due to the small size of this example, Filtering is omitted, while Clustering lies out of the scope of our work. Therefore, we exclusively focus on Entity Matching, where a typical batch algorithm considers all pairs of entities, returning $e_1 \equiv e_4$ and $e_2 \equiv e_7$ after processing the entire data sources, i.e., after examining 12 pairs. In contrast, Progressive Entity Matching defines a processing order, as in Figure 1(c), so as to promote the most likely matches and detect most of them even if the processing is terminated prematurely.

Most existing progressive methods are independent of matching decisions, defining a *static* processing order [34, 45, 54]. Yet, they have some limitations: (i) They disregard recent advances in Filtering: they exclusively consider (meta-)blocking as a pre-processing step for addressing volume, but recent studies show that nearest neighbor search achieves a much higher performance [25, 35, 50]. (ii) They disregard recent advances in natural language processing and the dominance of the semantic entity representations that stem from pre-trained language models in both ER tasks, i.e., blocking and matching. Instead, they exclusively rely on the syntactic representation of entity profiles [25, 49, 50, 52, 56]. (iii) There is no generic framework that unifies the traditional, syntactic- and blocking-based approaches with the latest, semantic-based ones that leverage nearest neighbor search.

To address these shortcomings, we propose a novel framework for Progressive Entity Matching that is generic enough to cover any type of entity representation and of Filtering methods. Our goal is not to combine the outcome of different techniques (e.g., the syntactic- and the semantic-based),

but to propose a set of generic, integrated steps that offer a series of options to researchers and practitioners, facilitating the construction of progressive pipelines that leverage diverse methods in a seamless way. To the best of our knowledge, no other framework offers this in a unifying way.

Our design space consists of four consecutive steps: (i) Filtering restricts the computational cost to the most similar pairs of candidates. (ii) Weighting assigns a similar score to each candidate pair that is proportional to the matching likelihood of the entity profiles comprising it. (iii) Scheduling leverages the similarity scores in order to define the optimal processing order that gives precedence to the duplicate pairs over the non-matching ones. (iv) Matching analytically examines each candidate pair to decide whether its entity profiles are matching or not.

We explain how the filtering step of our framework incorporates all the existing state-of-the-art approaches, from blocking- and sorting-based to those leveraging nearest neighbor search. For each approach, we discuss the corresponding weighting functions for syntactic and semantic representations. For scheduling, we propose four main algorithms based on the concept of the similarity graph [29], which includes a node for each input entity with an edge connecting each candidate pair. We stress that the existing Progressive Entity Matching approaches cover only a small portion of those generated by our framework. We also stress that the matching process lies outside the scope of this work, since it is an orthogonal issue [45, 54], with numerous recent state-of-the-art solutions based on Deep Learning [8, 49].

To test the performance of our framework, we perform a grid search to fine-tune a wide range of end-to-end pipelines over a set of 10 well-established real-world datasets for Record Linkage and of 8 for Deduplication. The experimental results indicate the best combination of filtering and scheduling algorithms in terms of effectiveness. We compare the best configuration of each filtering approach with the state-of-the-art from the literature, i.e., DeepBlocker [50] and Sparkly [35]. The former leverages semantic representations and the latter syntactic ones. This is the first time the two algorithms are applied on Progressive Entity Matching. Our results indicate that DeepBlocker consistently underperforms our semantic-based nearest-neighbor approach both with respect to effectiveness and to time efficiency. Sparkly is much faster than our syntactic-based nearest neighbor approach (due to its Spark-based parallelization), but at the same time, it is significantly less accurate. We also consider an additional state-of-the-art technique as baseline method: I-PES [11].

Overall, we make the following contributions:

(1) We introduce a generic framework for Progressive Entity Matching consisting of four steps, that organize the existing approaches, and giving rise to some new. Among the latter are the first progressive methods leveraging nearest-neighbor search.
(2) We analytically explain the configuration space of each progressive method generated by our framework. Some of the new methods are the first to apply pre-trained language models to Progressive Entity Matching.
(3) We perform an extensive experimental analysis of all Progressive Entity Matching approaches and configurations (using grid search) over 18 well-established datasets. Our experiments give valuable insights into the relative performance of the filtering and scheduling algorithms and demonstrate the superiority of our approaches over the state-of-the-art in the literature.

The implementation of our approaches is available online at: https://github.com/JacobMaciejewski/PER-Design-Space-Exploration.

2 Related Work

Progressive Entity Matching methods can be distinguished in two main categories, the *static* and the *dynamic* [29]. The former generate a processing order that is independent of the matching decisions,

The Battleship Approach to the Low Resource Entity Matching Problem

BAR GENOSSAR, Technion – Israel Institute of Technology, Israel
AVIGDOR GAL, Technion – Israel Institute of Technology, Israel
ROEE SHRAGA, Worcester Polytechnic Institute, USA

Entity matching, a core data integration problem, is the task of deciding whether two data tuples refer to the same real-world entity. Recent advances in deep learning methods, using pre-trained language models, were proposed for resolving entity matching. Although demonstrating unprecedented results, these solutions suffer from a major drawback as they require large amounts of labeled data for training, and, as such, are inadequate to be applied to *low resource entity matching* problems. To overcome the challenge of obtaining sufficient labeled data we offer a new *active learning* approach, focusing on a selection mechanism that exploits unique properties of entity matching. We argue that a distributed representation of a tuple pair indicates its informativeness when considered among other pairs. This is used consequently in our approach that iteratively utilizes space-aware considerations. Bringing it all together, we treat the low resource entity matching problem as a Battleship game, hunting indicative samples, focusing on positive ones, through awareness of the latent space along with careful planning of next sampling iterations. An extensive experimental analysis shows that the proposed algorithm outperforms state-of-the-art active learning solutions to low resource entity matching, and although using less samples, can be as successful as state-of-the-art fully trained known algorithms.

CCS Concepts: • **Information systems** → **Entity resolution**; • **Computing methodologies** → **Active learning settings**.

Additional Key Words and Phrases: entity resolution, entity matching, active learning

ACM Reference Format:
Bar Genossar, Avigdor Gal, and Roee Shraga. 2023. The Battleship Approach to the Low Resource Entity Matching Problem. *Proc. ACM Manag. Data* 1, 4 (SIGMOD), Article 224 (December 2023), 25 pages. https://doi.org/10.1145/3626711

1 INTRODUCTION

Entity matching, with variations as entity resolution, record linkage, record deduplication and more, is a core data integration task that plays a crucial role in any data project life cycle [8, 15]. Essentially, entity matching aims to identify dataset tuples that refer to the same real-world entities.

Traditional methods for entity matching and its variations focused on string similarity [20, 21, 27, 31] and probabilistic approaches [10], followed by rule-based methods [58, 59]. Learning-based approaches were also suggested [4, 26] with an increased focus on deep learning in recent years [13, 14, 24, 25, 28, 41, 68], culminating in the use of pre-trained language models, and specifically BERT [9], fine-tuned for entity matching [7, 29, 30, 45].

Authors' addresses: Bar Genossar, Technion – Israel Institute of Technology, Haifa, Israel, sbargen@campus.technion.ac.il, sbargen@campus.technion.ac.il; Avigdor Gal, Technion – Israel Institute of Technology, Haifa, Israel, avigal@technion.ac.il; Roee Shraga, Worcester Polytechnic Institute, Worcester, Massachusetts, USA, rshraga@wpi.edu.

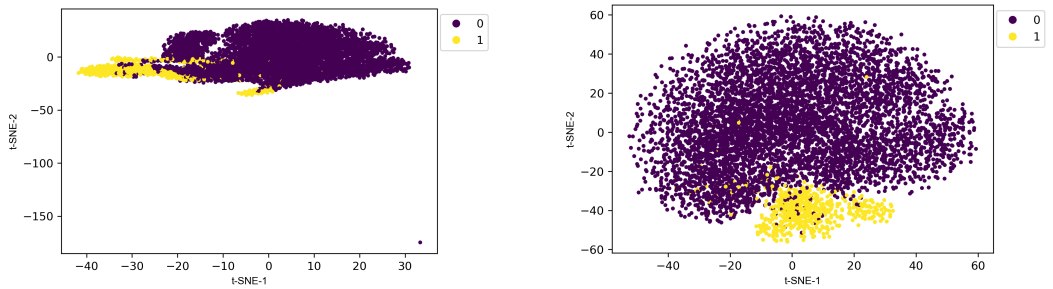
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/12-ART224 $15.00
https://doi.org/10.1145/3626711

Machine learning approaches to entity matching are by-and-large supervised, requiring labeled data for training, and in the case of deep learning, massive amounts of them [34, 46, 60]. Obtaining labeled data is challenging, mostly due to the need for intensive human expert labor that accompanies the labeling process [33, 34, 55]. This, in turn, calls for methods that reduce the labeling process load while maintaining an accurate trained matcher.

Several approaches were suggested over the years to overcome the dependence on large amounts of data, among which, unsupervised [3, 65, 66] and active learning methods [47, 53]. Active learning, the most prevalent approach to the insufficient labeled data problem, offers methods for training using a limited amount of labeled data. This is done by zeroing in on informative samples to be labeled and used for training. An active learning framework uses selection criteria (*e.g.*, variations of uncertainty [25, 38, 48] and centrality [67, 69]) to pinpoint the samples that are expected to improve the model performance the most. These selected samples are then sent to a labeling oracle and added to the available training set in following iterations.

A key tool which we use to quantify both uncertainty and centrality of a sample (a record pair in our case) is its vector-space latent representation. Traditional entity matching approaches constructed pair feature vector using widely used similarity measures [4, 26, 65] or learnable attribute-wise similarity vector using neural networks [24, 41]. Contemporary solutions [7, 29, 45], which demonstrate superior performance and do not require feature engineering, yield an embedding vector as alternative to feature vectors when representing the input (pair) with respect to the task.

In this work we offer a solution to low resource entity matching tasks, where there is a limited budget for labeling. In particular, we attend to the labeled data scarcity problem using active learning, proposing a novel strategy tailored to the specific characteristics of entity matching. Entity matching usually suffers from inherent label imbalance, where the number of negative (non-match) pairs overshadows the number of positive (match) ones [2]. When training with insufficient amount of data, this property is often problematic since the model may struggle to generalize to unseen minority class samples (in this case, matching pairs). To tackle this challenge, we ensure a balanced sampling of positive and negative samples using high-dimensional latent space encoding for tuple pairs. Specifically, we argue that vector-space similarity between pair representations reflects, with high probability, agreement of their labels. We next use this notion to carefully guarantee a balanced sampling, although positive pairs are harder to be found.



(a) Amazon-Google (b) Walmart-Amazon

Fig. 1. Visualization of pair distribution by t-SNE, partitioned into match and non-match pairs. Using representative vectors (with dimension of 768) of a fully trained models we demonstrate that match pairs tend to gather together.

Example 1. *Figure 1 provides an illustration of the latent space of tuple pairs over well known benchmarks using t-SNE [64]. Figure 1a presents the widely used Amazon-Google dataset,*[1] *which consists of 11,460 pairs, with only 1,167 (∼10%) labeled as match samples. To demonstrate the pairs behavior, we trained a DITTO [29] model with the fully available train set (60% of the entire dataset). After training, we extracted the latent representations (dimension of 768) generated for each tuple pair and applied t-SNE to create a human-readable 2-dimensional reduced space, where the yellow dots represent match pairs and the purple dots represent non-match pairs. As illustrated, there is a concentration of match pairs in a few main areas of the latent space, forming a cohesive and clear partition between match and non-match samples. As another example, Figure 1b presents the pairs distribution over the Walmart-Amazon dataset*[2] *(10,242 pairs, 962 matches, with 60% of the entire dataset used as a train set). Again, we see a concentration of match pairs, this time mainly in a single area of the reduced latent space.*

This example suggests that a latent space can identify regions where mostly match or non-match samples reside. We use this observation to tackle a major problem in active learning, namely modeling prediction uncertainty. A standard approach is to use the model prediction confidence score as a proxy to its underlying uncertainty. However, when using transformer-based models this method is likely to fail, as they tend to produce extreme confidence values (close to 0 or 1) which barely reflect the real confidence [17, 22]. We propose a new approach for modeling the uncertainty of a pair, based on its agreement with other pairs in its vicinity, such that when the correspondence between pair's prediction and its surroundings is higher, its uncertainty becomes lower.

1.1 Contributions

In this work we offer a novel active learning approach to entity matching, utilizing an effective space partitioning for diversity, and locality-based measurements for targeted sample selection. Specifically, equipped with the observed phenomenon of concentration of match pairs (Figure 1), we propose a sampling strategy that can be intuitively demonstrated using the popular Battleship game[3] (hence the battleship approach in the manuscript title). Our strategy uses a locality principle, searching in a vicinity of a match (or non-match) pair to create a balanced training set for human labeling. To balance locality with diversity and supporting model generalization we use a constrained version of the well-known k-means clustering algorithm [6], combined with centrality and uncertainty measures that offers an efficient selection mechanism of samples.

Our main contribution can be summarized as follows:

(1) A novel active learning sample selection method, based on match and non-match pair locality to ensure a balanced training set. The proposed solution uses tuple pair representation as a tool for diverse sampling.
(2) A novel uncertainty measure, overcoming the barrier of dichotomous confidence values assigned by transformer pre-trained language model.
(3) A large-scale empirical evaluation showing the effectiveness of our approach.
(4) An open source access to our implementation.[4]

The rest of the paper is organized as follows. We survey the necessary background for our task in Section 2, positioning entity matching in the context of active learning. In Section 3 we propose a new active learning algorithm for entity matching, inspired by the battleship game. We describe the experimental setup in Section 4 and present the empirical evaluation in Section 5. Then, we

[1] https://pages.cs.wisc.edu/~anhai/data1/deepmatcher_data/Structured/Amazon-Google/exp_data/
[2] https://pages.cs.wisc.edu/~anhai/data1/deepmatcher_data/Structured/Walmart-Amazon/exp_data/
[3] https://en.wikipedia.org/wiki/Battleship_(game)
[4] https://github.com/BarGenossar/The-Battleship-Approach-to-AL-of-EM-Problem

Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration

JIANHONG TU, Renmin University of China, China
JU FAN*, Renmin University of China, China
NAN TANG, QCRI / HKUST (GZ), Qatar / China
PENG WANG, Renmin University of China, China
GUOLIANG LI, Tsinghua University, China
XIAOYONG DU, Renmin University of China, China
XIAOFENG JIA, Beijing Big Data Centre, China
SONG GAO, Beijing Big Data Centre, China

Data matching – which decides whether two data elements (*e.g.,* string, tuple, column, or knowledge graph entity) are the "same" (*a.k.a.* a match) – is a key concept in data integration, such as entity matching and schema matching. The widely used practice is to build task-specific or even dataset-specific solutions, which are hard to generalize and disable the opportunities of knowledge sharing that can be learned from different datasets and multiple tasks. In this paper, we propose **Unicorn**, a unified model for generally supporting common data matching tasks. **Unicorn** can enable knowledge sharing by learning from multiple tasks and multiple datasets, and can also support zero-shot prediction for new tasks with zero labeled matching/non-matching pairs. However, building such a unified model is challenging due to heterogeneous formats of input data elements and various matching semantics of multiple tasks. To address the challenges, **Unicorn** employs one generic `Encoder` that converts any pair of data elements (a, b) into a learned representation, and uses a `Matcher`, which is a binary classifier, to decide whether a matches b. To align matching semantics of multiple tasks, **Unicorn** adopts a mixture-of-experts model that enhances the learned representation into a better representation. We conduct extensive experiments using 20 datasets on seven well-studied data matching tasks, and find that our unified model can achieve better performance on most tasks and on average, compared with the state-of-the-art specific models trained for ad-hoc tasks and datasets separately. Moreover, **Unicorn** can also well serve new matching tasks with zero-shot learning.

CCS Concepts: • **Information systems → Mediators and data integration**.

Additional Key Words and Phrases: data matching, data integration, multi-task learning

ACM Reference Format:
Jianhong Tu, Ju Fan, Nan Tang, Peng Wang, Guoliang Li, Xiaoyong Du, Xiaofeng Jia, and Song Gao. 2023. Unicorn: A Unified Multi-tasking Model for Supporting Matching Tasks in Data Integration. *Proc. ACM Manag. Data* 1, 1, Article 84 (May 2023), 26 pages. https://doi.org/10.1145/3588938

*Ju Fan is the corresponding author.

Authors' addresses: Jianhong Tu, Renmin University of China, Beijing, China, tujh@ruc.edu.cn; Ju Fan, Renmin University of China, Beijing, China, fanj@ruc.edu.cn; Nan Tang, QCRI / HKUST (GZ), Doha / Guangzhou, Qatar / China, ntang@hbku.edu.qa; Peng Wang, Renmin University of China, Beijing, China, lisa_wang@ruc.edu.cn; Guoliang Li, Tsinghua University, Beijing, China, liguoliang@tsinghua.edu.cn; Xiaoyong Du, Renmin University of China, Beijing, China, duyong@ruc.edu.cn; Xiaofeng Jia, Beijing Big Data Centre, Beijing, China, jiaxf@jxj.beijing.gov.cn; Song Gao, Beijing Big Data Centre, Beijing, China, gaos@jxj.beijing.gov.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.
2836-6573/2023/5-ART84 $15.00
https://doi.org/10.1145/3588938

1 INTRODUCTION

Data matching generally refers to the process of deciding whether two data elements are the "same" (*a.k.a.* a match) or not, where each data element could be of different classes such as string, tuple, column, and so on. Data matching is a key concept in data integration [13] and data preparation [9] that includes a wide spectrum of tasks. In this paper, we consider seven common data matching tasks, namely entity matching, entity linking, entity alignment, string matching, column type annotation, schema matching, and ontology matching, as shown in Figure 1.

Due to their importance, almost all the aforementioned matching tasks have been studied for decades, and still remain to be important research topics. With the tremendous successes of deep learning, deep neural networks have been widely used for tackling various matching tasks, such as DeepMatcher [34] and Ditto [30] for entity matching (see a recent tutorial [4]), and TURL [10] and HNN [5] for column type annotation. However, these current deep learning based solutions are task-specific or even dataset-specific, which are referred to as **specific models** in this paper.

There are two main **limitations of specific models**. First, they can only learn knowledge from specific tasks or datasets. That is, the learned knowledge cannot be shared across different models. For example, the knowledge learned by a column type annotation model (*e.g.,* TURL [10]) cannot be shared with an entity matching model (*e.g.,* Ditto [30]) due to different neural network designs, although very likely the two models can help each other. Second, one model has to be trained (or fine-tuned) only on the labeled examples of each task or dataset, which is inefficient and has a high monetary cost. For example, DeepMatcher [34] and Ditto [30] have to be fine-tuned on a new entity matching dataset with at least hundreds of labeled matching/non-matching pairs.

In this paper, we propose **`Unicorn`**, a unified model to support multiple data matching tasks, as shown in Figure 1. Compared with aforementioned specific models, **`Unicorn`** has the following notable advantages.

(1) ***Task unification*** that generalizes task-specific solutions into one unified model, thus achieving lower maintenance complexity and smaller model sizes, compared with specific models.
(2) ***Multi-task learning*** that enables the unified model to learn from multiple tasks and multiple datasets to make full use of *knowledge sharing*, which even outperforms specific models trained only on their own datasets separately.
(3) ***Zero-shot prediction*** that allows the model to make predictions for a new task or a new dataset with *zero* labeled matching/non-matching pairs.

Although several unified models have been recently studied in the NLP and CV communities [3, 41, 42], building such a unified model for supporting multiple data matching tasks is challenging.

(1) Data elements in the matching tasks can take ***heterogeneous formats***, such as tuples and columns in tabular data, entities in knowledge graphs, and plain text, which will increase the difficulty of task unification.
(2) Each data matching task may have its ***unique matching semantics***. For example, entity matching that matches two tuples is different from schema matching that matches two columns. Thus, it is non-trivial to enable knowledge sharing among multiple tasks.

To address the challenges, we develop a general framework **`Unicorn`**, which consists of three key modules: an `Encoder`, a `Mixture-of-Experts` layer, and a `Matcher` (see Figure 1). First, the `Encoder` converts any pair of elements (a, b) with heterogeneous formats into a learned representation. To this end, we propose to serialize any pair of elements into text while still preserving their inherent structure, and employ a unified pre-trained language model for effective encoding. Second, to align matching semantics of various tasks or datasets, the `Mixture-of-Experts` layer enhances the learned representation into a better representation by combining the knowledge from multiple "experts" controlled by a learned gating network. Third, the `Matcher` is a typical binary classifier,

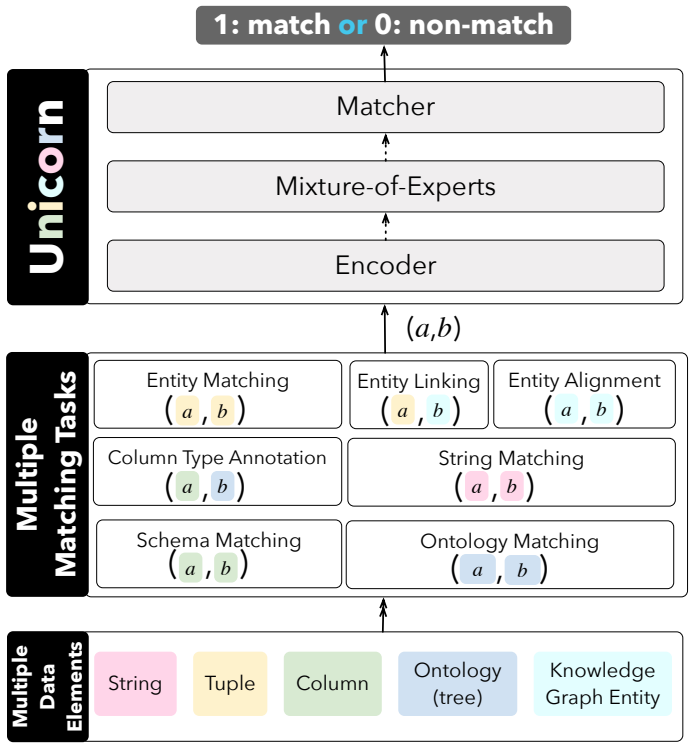


Fig. 1. **Unicorn** is a unified model for "data matching" task in data integration. So far, it supports seven matching tasks in data integration for a pair (a, b) where a or b may be of the same or different types of data elements. **Unicorn** consists of a unified `Encoder`, a `Mixture-of-Experts` layer, and a `Matcher` that will decide whether (a, b) is a match (1) or a non-match (0).

e.g., a multilayer perceptron (MLP), to predict either 1 (for match) or 0 (for non-match) by taking the above representation as input.

Our notable contributions are summarized as follows.

(1) As far as we know, **Unicorn** is the first unified multi-tasking model towards supporting various matching tasks in data integration. We formally define the problem of data matching tasks and introduce an overview of the **Unicorn** framework (Section 2).
(2) We develop effective techniques for two key modules in the **Unicorn** framework. We introduce a unified representation learning method for the `Encoder` module (Section 3) and design effective methods for the `Mixture-of-Experts` layer (Section 4).
(3) We conduct a thorough evaluation on 20 datasets for the seven common data matching tasks shown in Figure 1. Extensive experiments show that **Unicorn**, as a unified model, outperforms the state-of-the-art specific models on most tasks and on average (Section 5 **Exp-3**). In addition, **Unicorn** can well serve new matching tasks with zero-shot learning (Section 5 **Exp-4**).
(4) We make a unified benchmark for multiple data matching tasks available at Github[1] . We standardize the format of datasets from different data matching tasks, making these datasets more convenient to use. We publish the source code of **Unicorn**, and provide the trained **Unicorn**

[1]https://github.com/ruc-datalab/Unicorn